

Universidad de San Carlos de Guatemala

Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería

Introducción a la programación y computación 2, 2do. Semestre 2026

SISTEMA INTELIGENTE DE GESTIÓN DE LIBRERÍA MEDIANTE TIPOS DE DATO ABSTRACTO NO LINEALES Y PROGRAMACIÓN ORIENTADA A OBJETOS

202300345 – Horacio José Lemus Román

Resumen

El presente ensayo detalla el diseño y la implementación de un sistema inteligente para la administración de un catálogo de más de 100,000 libros distribuidos en diversas sucursales, dando solución a las deficiencias de tiempos de búsqueda derivadas del uso de listas extensas tradicionales. Se aborda la construcción de la solución utilizando exclusivamente Tipos de Dato Abstracto (TDAs) propios mediante el lenguaje C#, eliminando cualquier dependencia de estructuras nativas. A nivel técnico, se expone la implementación de un Árbol Binario de Búsqueda (BST) para garantizar tiempos logarítmicos en la localización de libros mediante su código ISBN único. Paralelamente, se detalla el uso de listas enlazadas simples para modelar la organización jerárquica de categorías y subcategorías, emulando la distribución de una biblioteca física. El impacto principal de este desarrollo radica en la optimización algorítmica y la eficiencia en la gestión de memoria dinámica, demostrando que una abstracción de datos adecuada mejora sustancialmente la recuperación de información y la experiencia del usuario.

Palabras clave

Catálogo, Árbol-Binario, Listas-Enlazadas, TDAs, Abstracción.

Abstract

This essay details the design and implementation of an intelligent system for managing a catalog of over 100,000 books distributed across various branches, solving search time deficiencies derived from traditional extensive lists. It discusses the construction of the solution using exclusively custom Abstract Data Types (ADTs) in C#, removing any reliance on native structures. Technically, it explains the implementation of a Binary Search Tree (BST) to guarantee logarithmic times in locating books via their unique ISBN. Concurrently, it details the use of simple linked lists to model the hierarchical organization of categories and subcategories, emulating a physical library's layout. The primary impact of this development lies in algorithmic optimization and dynamic memory management efficiency, proving that proper data abstraction substantially improves information retrieval and user experience.

Keywords

Catalog, Binary-Tree, Linked-Lists, ADTs, Abstraction.

Introducción

La administración eficiente de grandes volúmenes de información es uno de los pilares de la ingeniería de software moderna. En el contexto de una cadena de librerías con un inventario masivo, el almacenamiento de registros en estructuras lineales planas genera cuellos de botella severos al momento de realizar consultas, afectando la operatividad del negocio. El propósito de este ensayo es exponer cómo la aplicación rigurosa de la Programación Orientada a Objetos y la construcción de Tipos de Dato Abstracto (TDAs) personalizados resuelven esta problemática. Mediante el desarrollo de un sistema jerárquico, se evidencia el aporte de estructurar los datos emulando el modelo físico de una biblioteca, separando la información en categorías y optimizando las búsquedas individuales. A través del documento, se responderá cómo la elección correcta de algoritmos no lineales transforma radicalmente el rendimiento de las operaciones de búsqueda y organización.

Desarrollo del tema

El núcleo del proyecto requirió abandonar las colecciones dinámicas provistas por el entorno de C# para construir desde cero los mecanismos de almacenamiento y recorrido en memoria dinámica, garantizando un control absoluto sobre la complejidad computacional.

a. Modelado Jerárquico de Categorías (Estructuras Dinámicas Lineales)

Para representar las relaciones jerárquicas similares a las de un organigrama o un sistema de carpetas, se diseñó la clase `Categoria`. Esta entidad actúa como un nodo complejo que gestiona dos listas enlazadas independientes: una para almacenar subcategorías (`NodoCategoria`) y otra para asociar los libros pertenecientes a dicho sector (`NodoLibroEnCategoria`). Esta separación permite que las categorías se presenten ordenadas alfabéticamente desde el momento de la inserción, aplicando comparaciones de cadenas (`string.Compare`) antes de reasignar los punteros, lo que agiliza la generación de la vista jerárquica sin necesidad de métodos de ordenamiento posteriores de alto costo computacional.

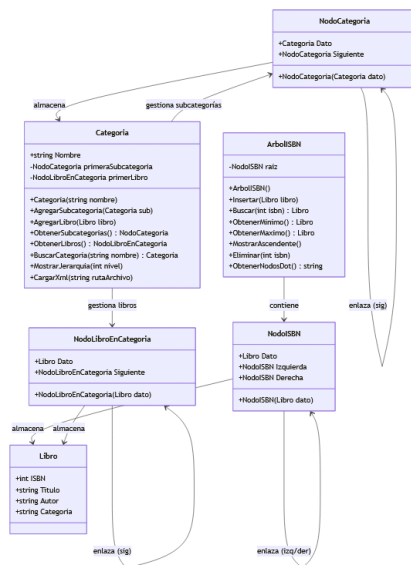


Figura 1. Diagrama de Clases del modelo jerárquico.

Fuente: elaboración propia.

b. Optimización de Búsqueda (Árbol Binario de Búsqueda)

El requerimiento crítico de mantener tiempos de búsqueda reducidos al consultar por código ISBN (identificador único numérico) hizo inviable el uso de listas enlazadas para el inventario global. Se implementó un Árbol Binario de Búsqueda (BST) a través de las clases `ArbolISBN` y `NodoISBN`. La naturaleza no lineal de esta estructura garantiza que, en promedio, las operaciones de inserción, búsqueda y eliminación posean una complejidad de $O(\log n)$, descartando mitades del catálogo en cada iteración recursiva.

c. Operaciones de recorrido y mantenimiento del catálogo

El sistema requería mostrar los libros en orden ascendente de ISBN, así como identificar los valores extremos. Gracias a la topología del BST, la obtención del libro con menor y mayor código se resuelve iterando hacia el puntero Izquierda o Derecha respectivo hasta encontrar una referencia nula. Para la impresión ascendente, se aplicó un recorrido *In-Order*, visitando primero el subárbol izquierdo, luego la raíz y finalmente el subárbol derecho, cumpliendo con la presentación de datos solicitada de manera natural y óptima.



Figura 2. Interfaz web de la plataforma.

Fuente: elaboración propia

Figura 3. Interfaz web de la plataforma.

Fuente: elaboración propia

Conclusiones

- La restricción de omitir las estructuras de datos nativas del lenguaje demostró que el diseño de Tipos de Dato Abstracto (TDAs) propios brinda un control granular sobre el consumo de memoria dinámica y el comportamiento de los punteros, habilidades esenciales para la optimización algorítmica.
- La implementación del Árbol Binario de Búsqueda fue la propuesta técnica definitiva para mitigar la lentitud del sistema original, demostrando que la estructura de los datos condiciona directamente el desempeño de las operaciones más críticas del negocio.
- El modelado orientado a objetos permitió abstraer un problema del mundo real (la organización física de estanterías en una biblioteca) y traducirlo eficientemente a listas enlazadas interconectadas, generando un código escalable y modular.

- Schildt, H. (2010). *C# 4.0 The Complete Reference*. McGraw-Hill Education.

Referencias bibliográficas

- Joyanes Aguilar, L. (2008). *Fundamentos de Programación: Algoritmos, estructura de datos y objetos* (4ta ed.). McGraw-Hill.